

Tier 3. Module 2 - Foundations of Cloud Computing

Final assignment

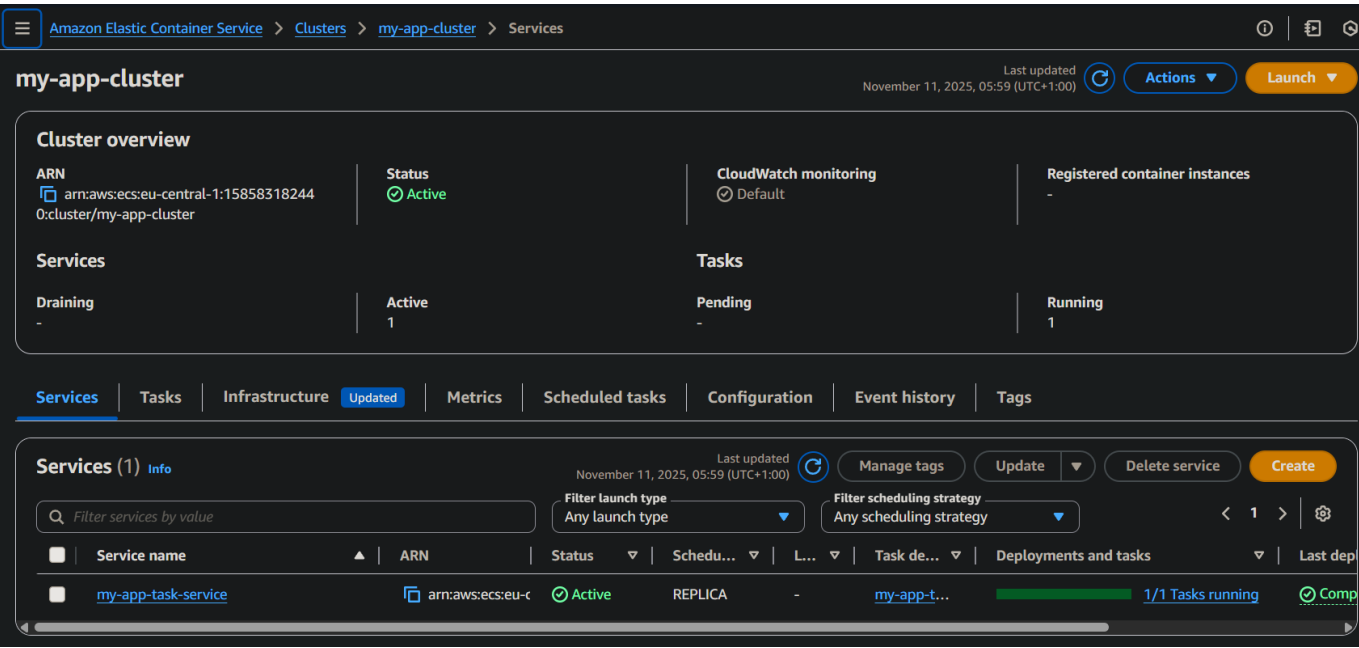
Technical Task

1. Choose a deployment environment

You can choose from:

- Elastic Beanstalk for automated web application deployment.
- Amazon ECS for containerized applications managed via AWS Fargate or EC2.
- Amazon EKS for scalable Kubernetes applications.
- Amazon EC2 for manual server management.

For the final project, the deployment of a containerized Django web application with a simple service that guesses objects in photos that can be uploaded to this service was chosen on the AWS cloud. The ECS service (serverless Fagate architecture) is best suited for this task, as its use greatly simplifies the deployment, control, and scaling of Dockerized applications.



2. Infrastructure and security

The following are the requirements:

- Set up VPC, public and private subnets.

Since this is a small training task and the application does not need to receive significant traffic from all over the world, only two availability zones with two public and two private subnets are sufficient to limit the budget.

VPC > Your VPCs > vpc-06a94e8f5313d494d

vpc-06a94e8f5313d494d / my-app-vpc

Actions

DetailsInfo

VPC ID

vpc-06a94e8f5313d494d

DNS resolution

Enabled

Main network ACL

acl-0f6ae3ff8f93577b9

IPv6 CIDR (Network border group)

-

State

Available

Tenancy

default

Default VPC

No

Network Address Usage metrics

Disabled

Block Public Access

Off

DHCP option set

dopt-0b1b75a2a997380fe

IPv4 CIDR

10.0.0.0/16

Route 53 Resolver DNS Firewall rule groups

-

DNS hostnames

Disabled

Main route table

rtb-0189c97182477720d

IPv6 pool

-

Owner ID

158583182440

Resource mapCIDRsFlow logsTagsIntegrations

Resource mapInfo

Show all details

VPC

Your AWS virtual network

my-app-vpc

Subnets (4)

Subnets within this VPC

eu-central-1a

Public-1

Private-1

eu-central-1b

Public-2

Private-2

Route tables (3)

Route network traffic to resources

my-app-public-rt

rtb-0189c97182477720d

my-app-private-rt

Network Connections

Connections to other resources

my-app-gateway

my-app-nat

VPC > Subnets

Subnets (1/4)Info

Last updated 2 minutes ago

Actions

Create subnet

Find subnets by attribute or tag

	Name	Subnet ID	State	VPC	Block Public...	IPv4 CIDR
☐	Private-2	subnet-061e7590f5f5481e4	Available	vpc-06a94e8f5313d494d my-...	Off	10.0.11.0/24
☐	Private-1	subnet-00f2fcfc9b761982	Available	vpc-06a94e8f5313d494d my-...	Off	10.0.10.0/24
☒	Public-1	subnet-0b73b6645a2c4728f	Available	vpc-06a94e8f5313d494d my-...	Off	10.0.0.0/24
☐	Public-2	subnet-05ca8eed62a1ee8c3	Available	vpc-06a94e8f5313d494d my-...	Off	10.0.1.0/24

subnet-0b73b6645a2c4728f / Public-1

DetailsFlow logsRoute tableNetwork ACLCIDR reservationsSharingTags

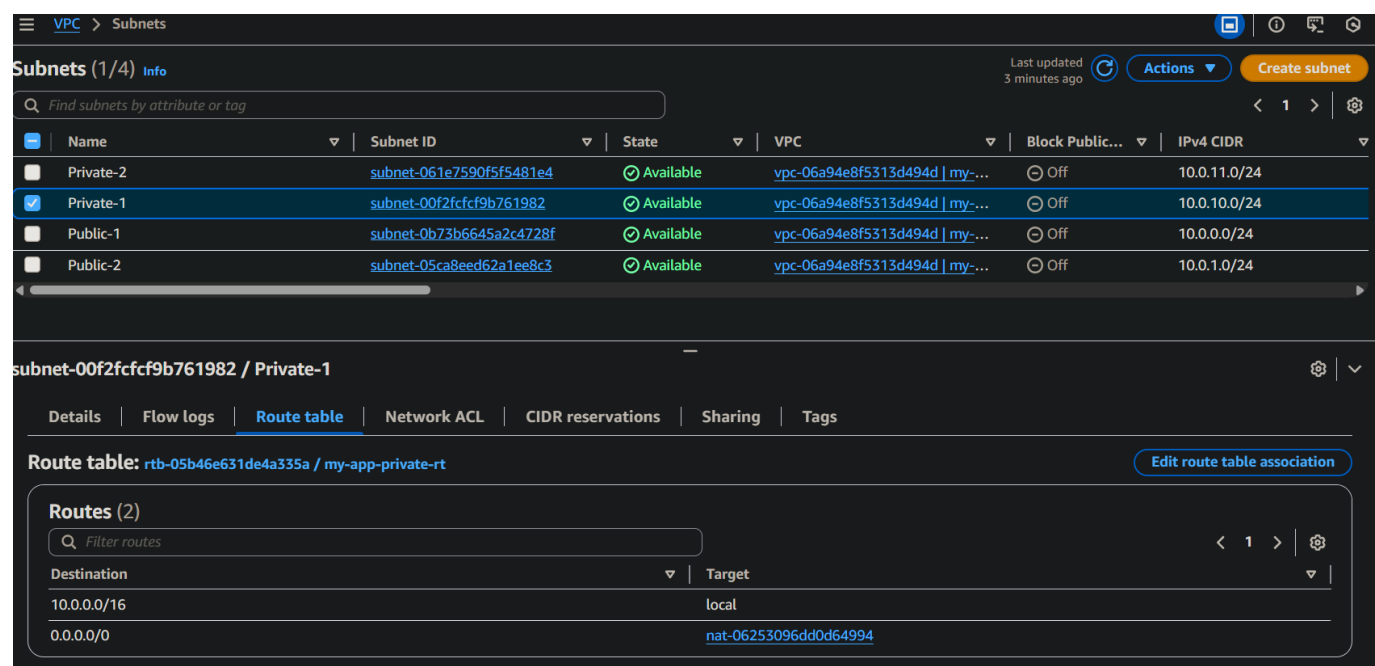
Route table: rtb-0c6488ddc7643ef0c / my-app-public-rt

Edit route table association

Routes (2)

Filter routes

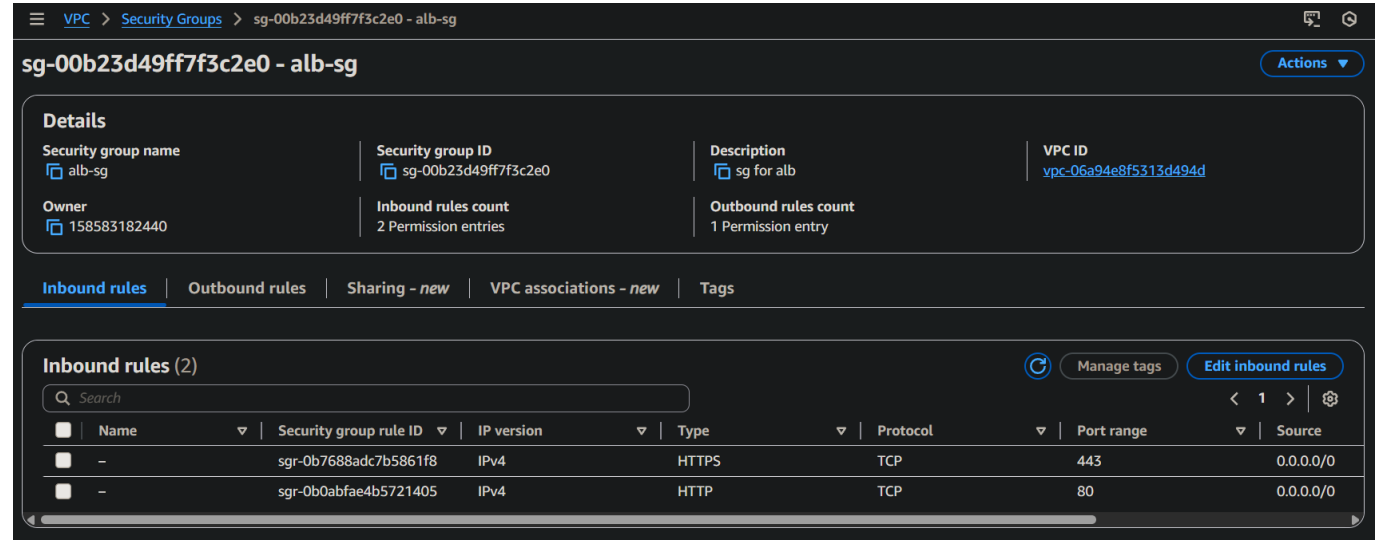
Destination	Target
10.0.0.0/16	local
0.0.0.0/0	igw-01520486ccb052689



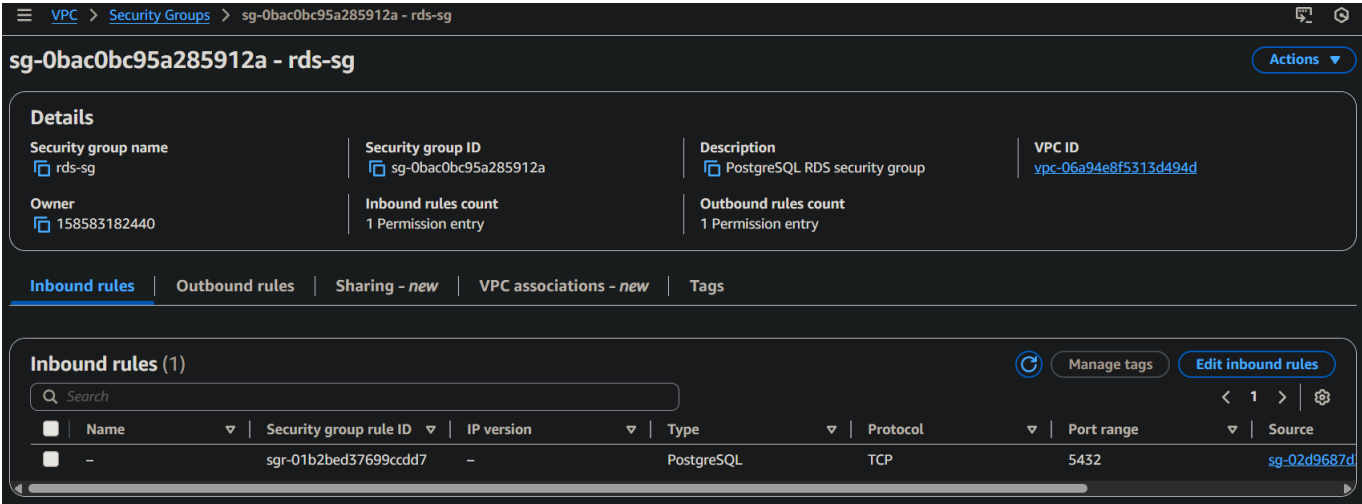
- Set up security groups to control traffic.

Security groups for public and private subnets.

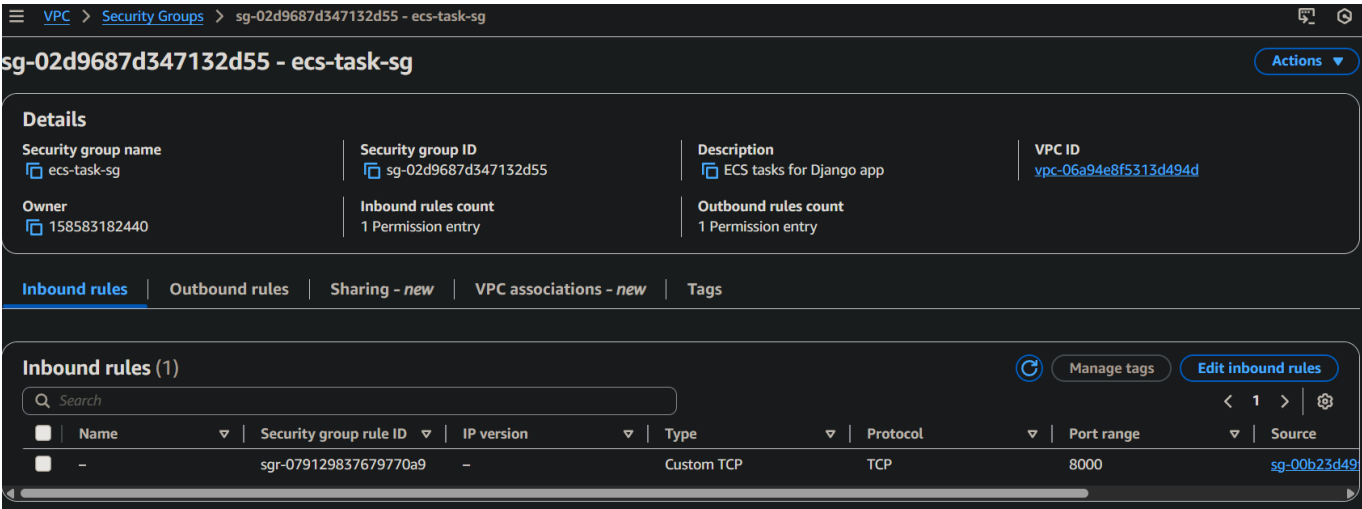
The security group for the ALB is the only one accessible from the Internet. All others are only available from the cloud.



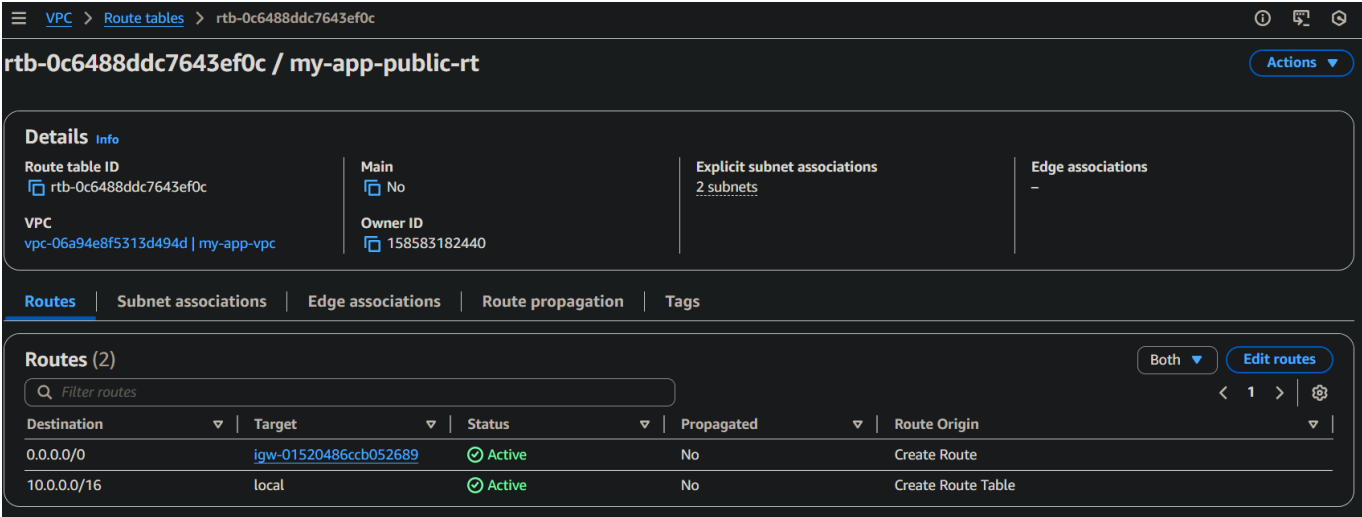
The security group for RDS allows inbound traffic on the default PostgreSQL port.



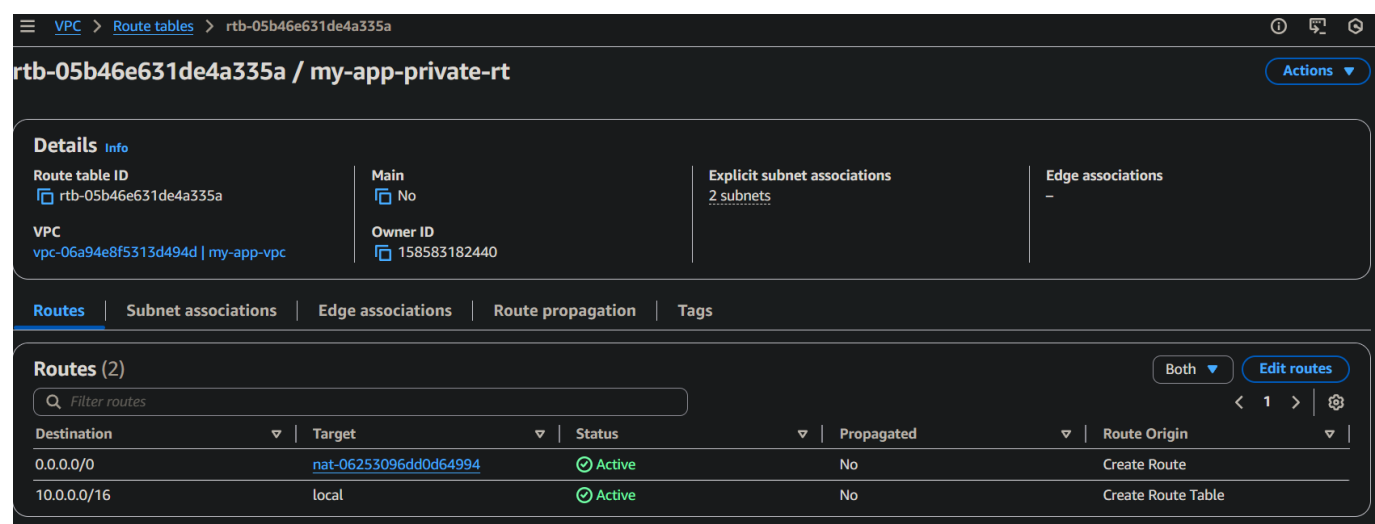
The web application in the container will be accessible on port 8000.



Route table for public subnets with an Internet gateway.

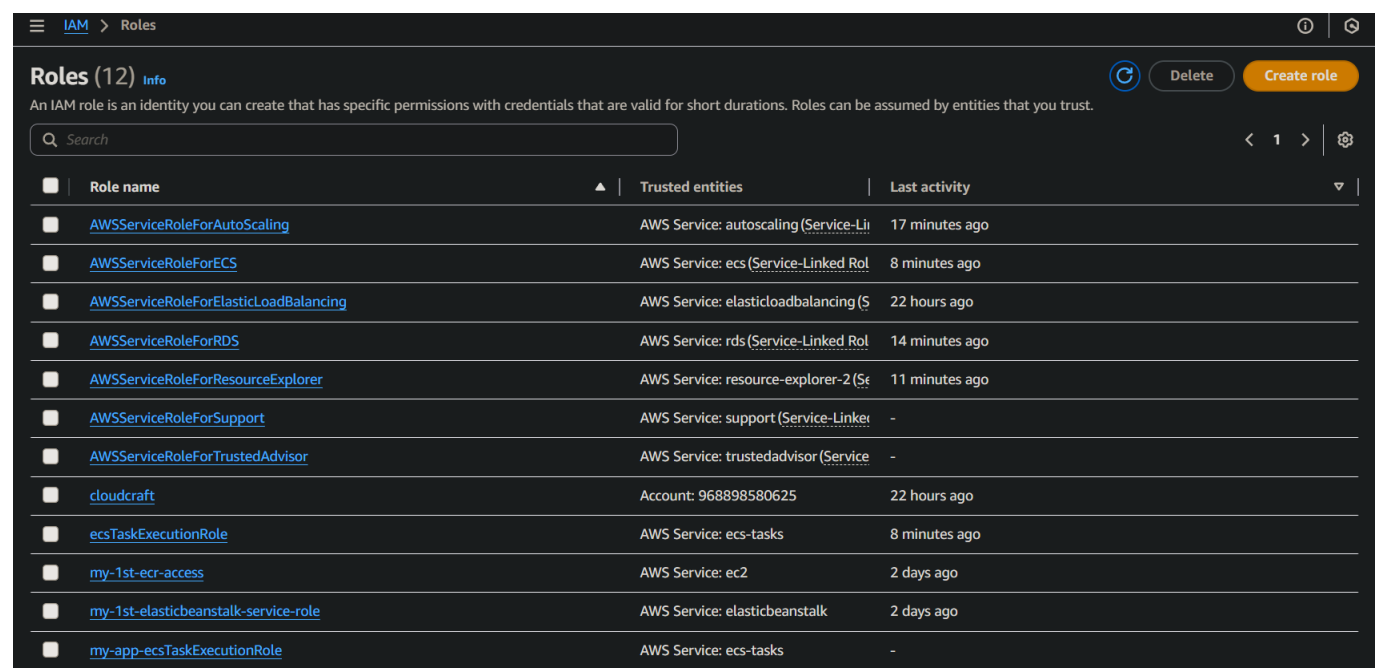


Route table for private subnets with an NAT gateway.



- Set up IAM roles to access AWS services.

IAM roles required for application access, operation, and read-only Cloudcraft access to automatically create network diagrams.

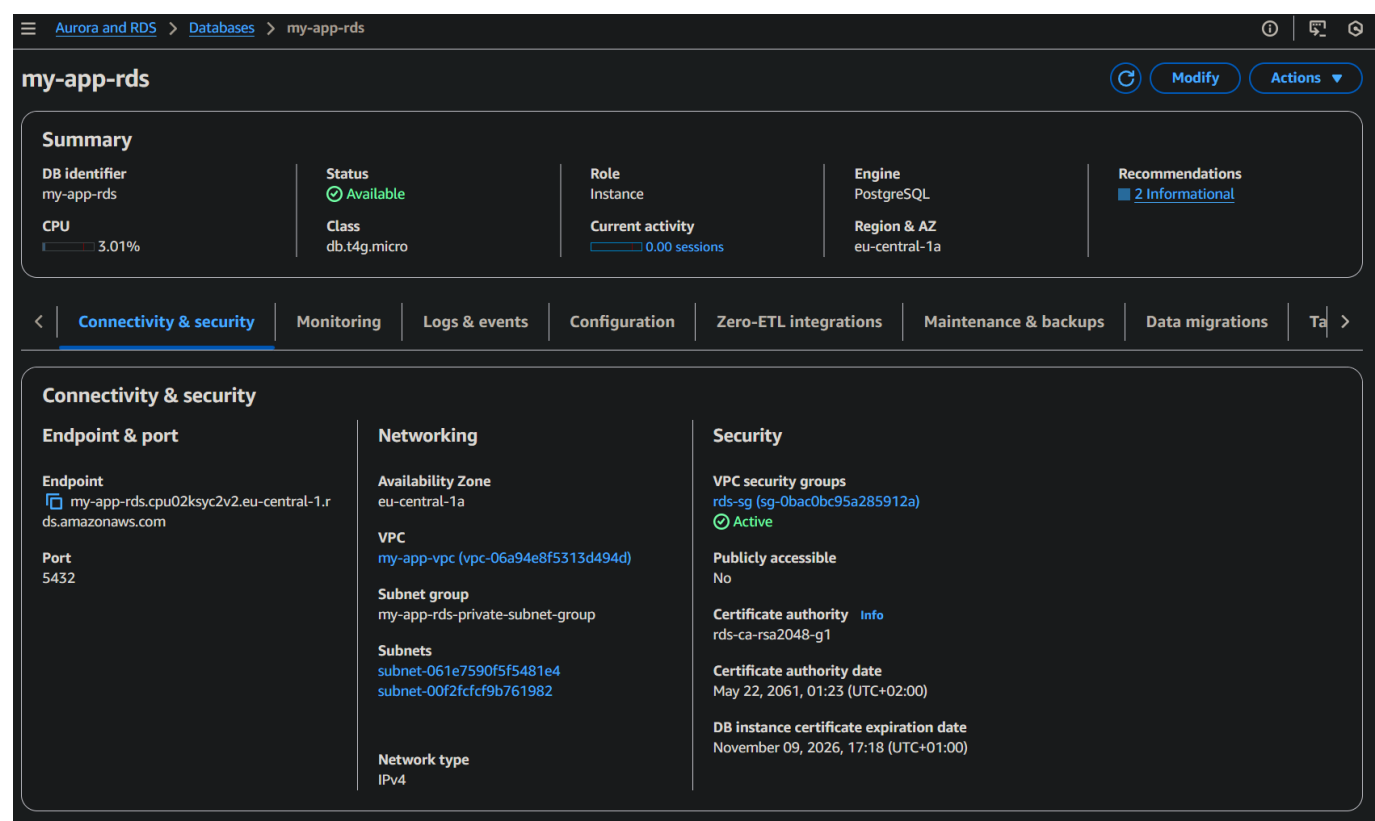


3. Database

Choose from the following:

- Using Amazon RDS for relational databases (MySQL, PostgreSQL).

The Django web application requires a PostgreSQL RDS database to store user roles and image object guessing results.

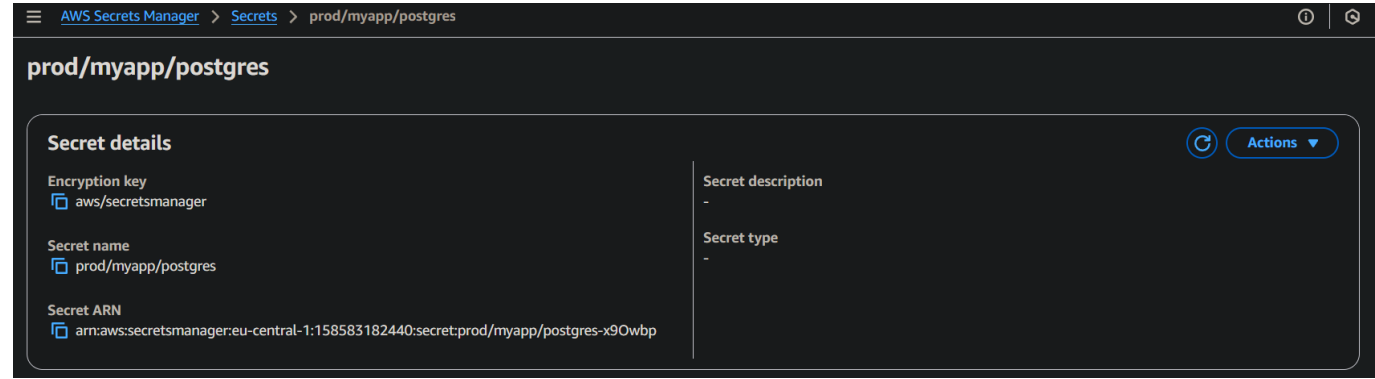


- Using Amazon DynamoDB for NoSQL storage.

Amazon DynamoDB is not required for this type of application.

- Optimize connections and protect passwords.

Secret manager for storing RDB access credentials.

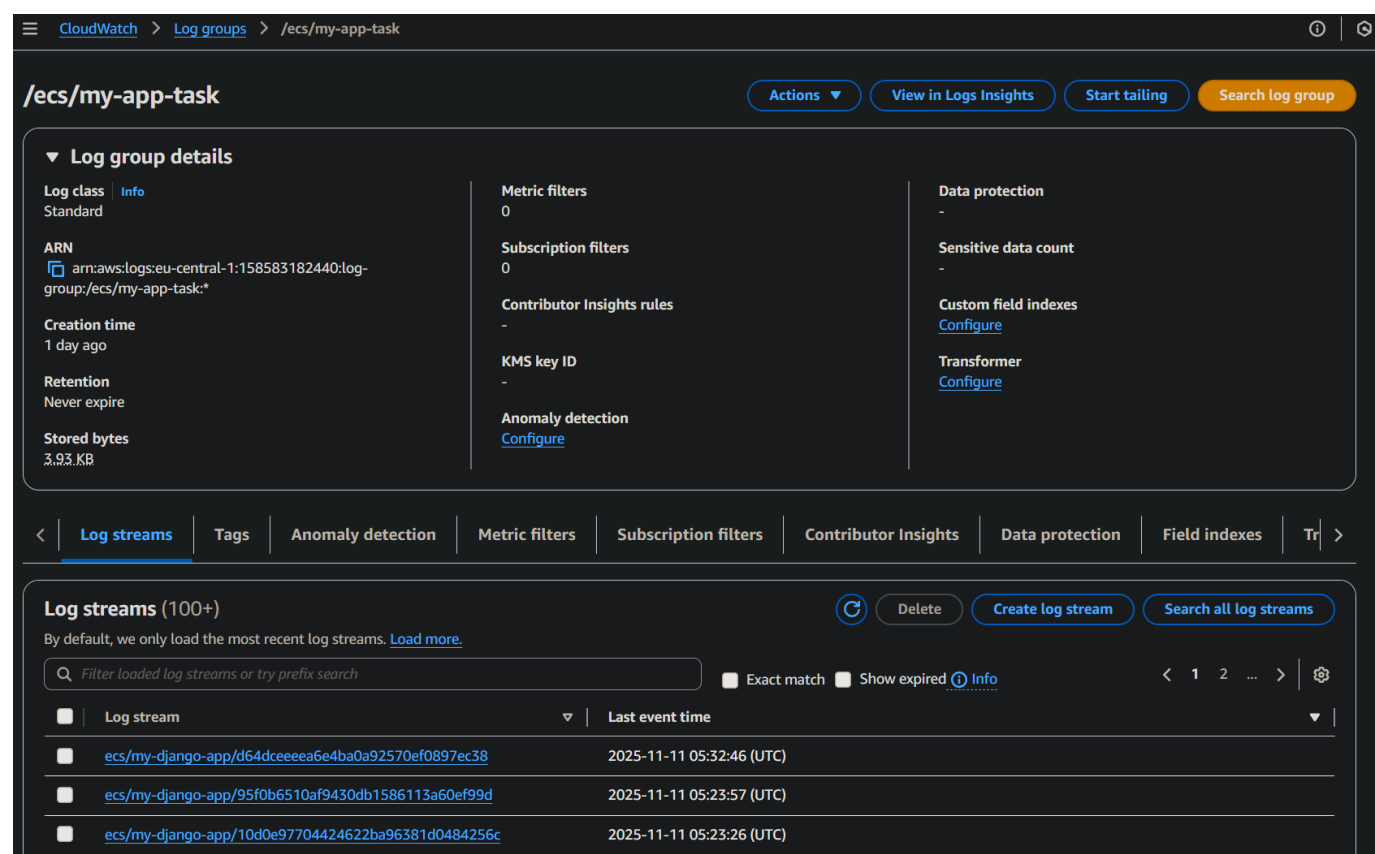


4. Monitoring and Scaling

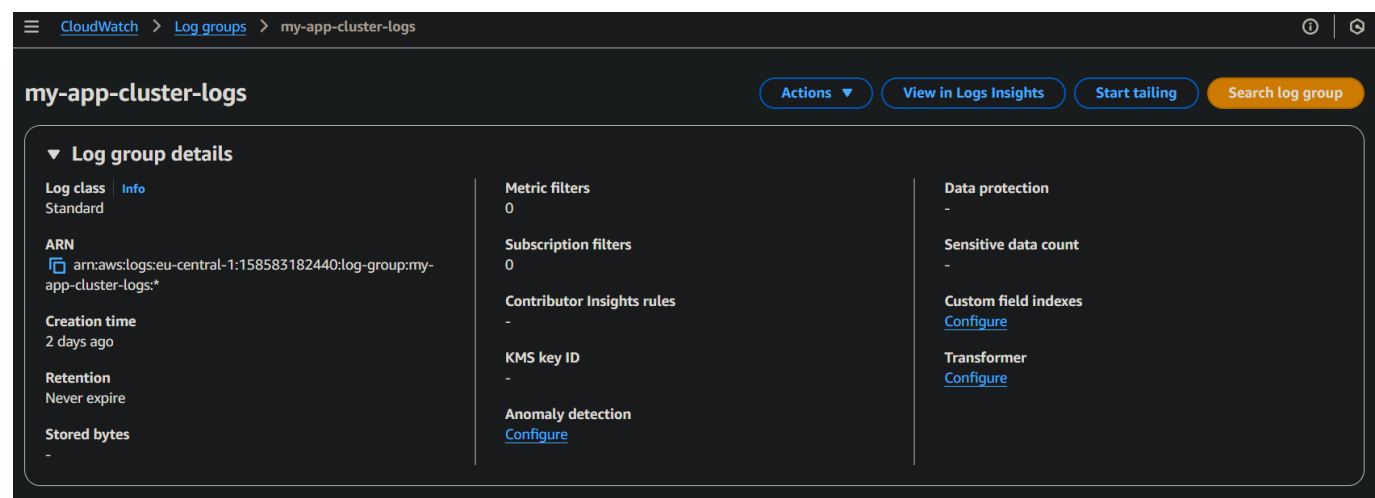
Required:

- Use Amazon CloudWatch to monitor system health.

Log group for monitoring ECS tasks.

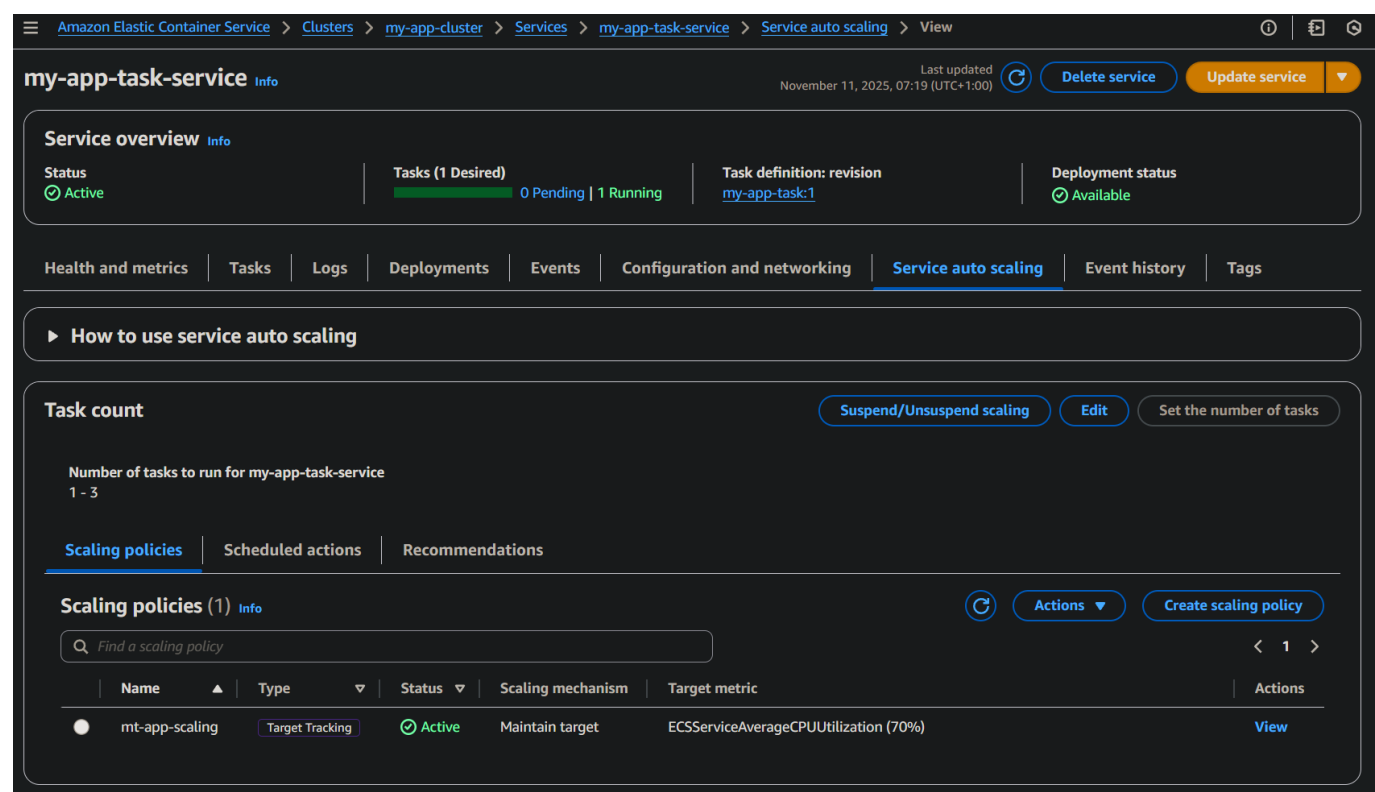


Log group for monitoring ECS cluster.



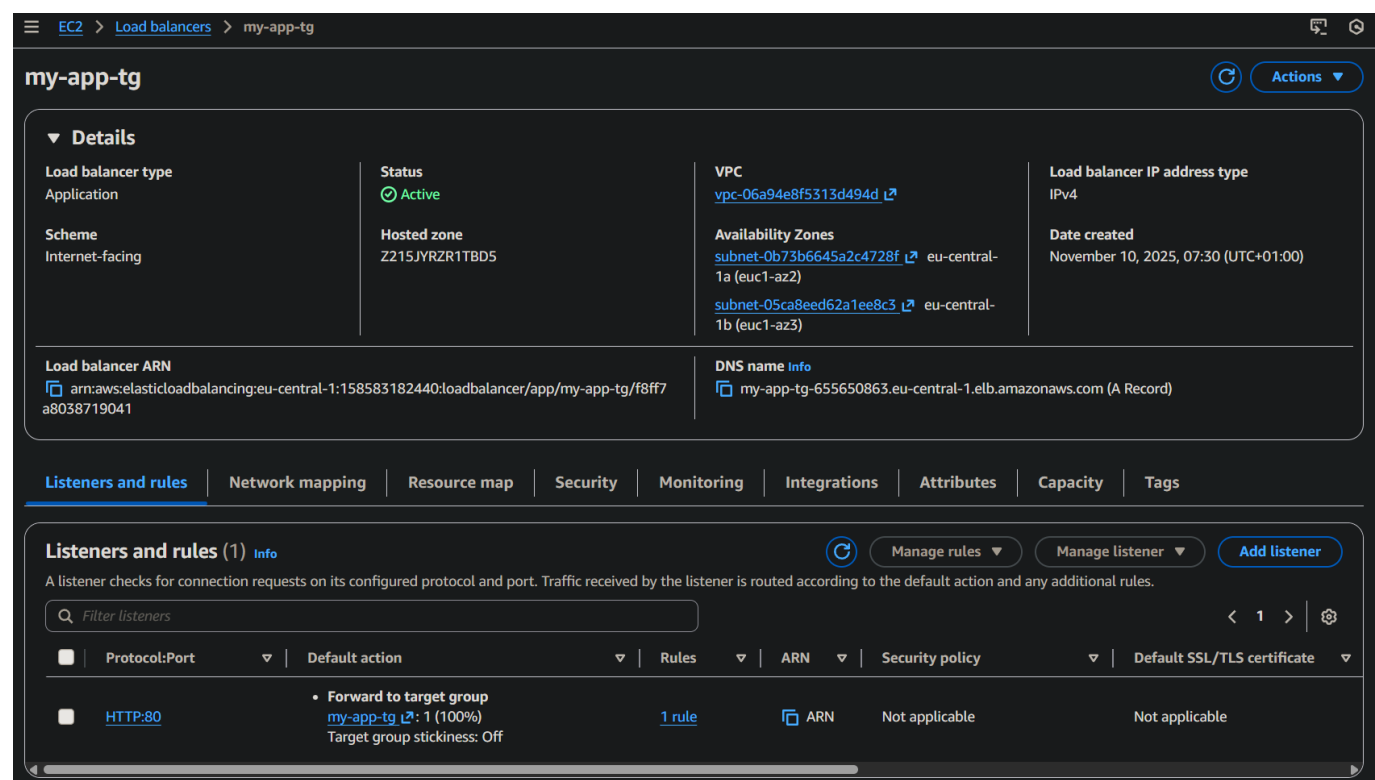
- Set up AWS Auto Scaling.

Auto scaling based on 70% SCU utilization.



- Add a load balancer (ALB or NLB).

Application load balancer Internet-facing and accesible on port 80.



Target group for ALB.

EC2 > Target groups > my-app-tg

my-app-tg

Actions

Details

arn:aws:elasticloadbalancing:eu-central-1:158583182440:targetgroup/my-app-tg:e58d29735326f355

Target type

IP

Protocol : Port

HTTP: 8000

Protocol version

HTTP1

VPC

vpc-06a94e8f5313d494d

IP address type

IPv4

Load balancer

my-app-tg

2

Total targets

2

Healthy

0

Unhealthy

0

Unused

0

Initial

0

Draining

0

Anomalous

Distribution of targets by Availability Zone (AZ)

Select values in this table to see corresponding filters applied to the Registered targets table below.

Targets

Monitoring

Health checks

Attributes

Tags

Registered targets (2)

Anomaly mitigation: Not applicable

Deregister

Register targets

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

Filter targets

	IP address	Port	Zone	Health status	Health status details	Admini...	Overri...	Anomaly detection result
	10.0.10.241	8000	eu-central-...	Healthy	Target deregistration i...	No override.	No overri...	Normal
	10.0.11.115	8000	eu-central-...	Healthy	Target deregistration i...	No override.	No overri...	Normal

5. Cost Optimization

You will be expected to:

- Use AWS Cost Explorer to analyze costs.

Billing and Cost Management > Cost Explorer > Monthly costs by service

Monthly costs by service

Report parameters

Recent reports

Save to report library

Cost and usage overview

Total cost

-\$0.00

Average daily cost

-\$0.00

Service count

16

Cost and usage graph

Costs (\$)

Nov-04* Nov-05* Nov-06* Nov-07* Nov-08* Nov-09* Nov-10*

Data Transfer Elastic Container Service EC2-Other Elastic Load Balancing Secrets Manager EC2 Container Registry (ECR) Relational Database Service CloudFormation Glue Others

Report parameters

Time

Standard Compare

Date Range

2025-11-04 — 2025-11-10

Displaying last 7 days

Granularity

Daily

Group by

Dimension

Service

Filters

Applied filters (1)

Clear all

Service

Choose services

Linked account

Choose linked accounts

Region

Regions included (1)

Europe (Frankfurt)

Instance type

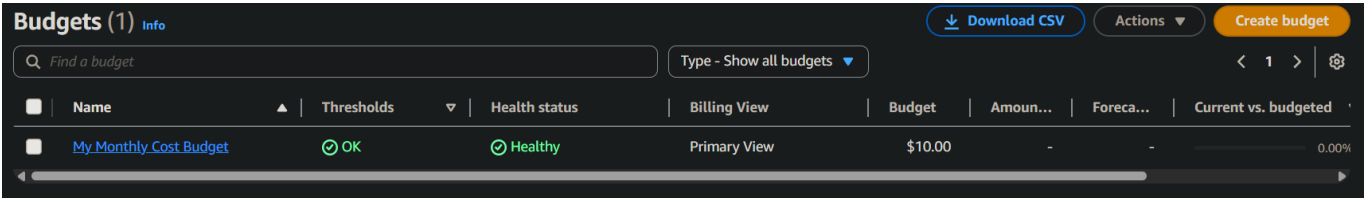
Choose instance types

Usage type

Choose usage types

9 / 13

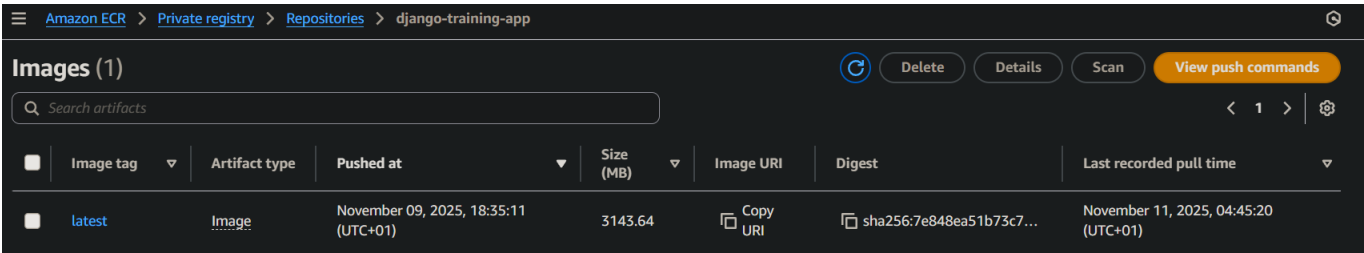
- Set up AWS Budgets to control costs.



Steps to Complete

1. Design your solution architecture
 - Draw a diagram that shows how different AWS services interact with each other.
 - Determine which technology stack will be used. Justify your choice.
2. Deploy the infrastructure in AWS
 - Use the AWS Console to create resources.
3. Deploy the web application
 - If using containerization, upload images to Amazon ECR.

ECR with latest image. The application source code is available under the [Link](#).

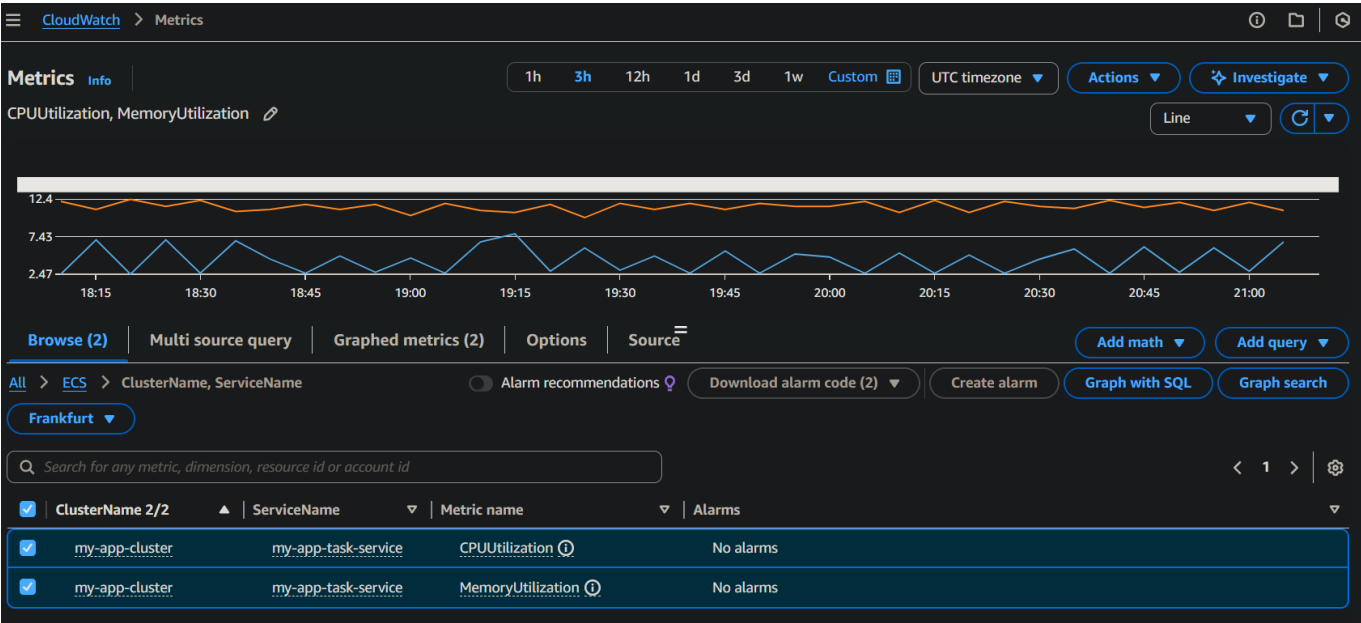


- If using EC2 or Elastic Beanstalk, ensure that the code runs correctly.

Not required for this type of application.

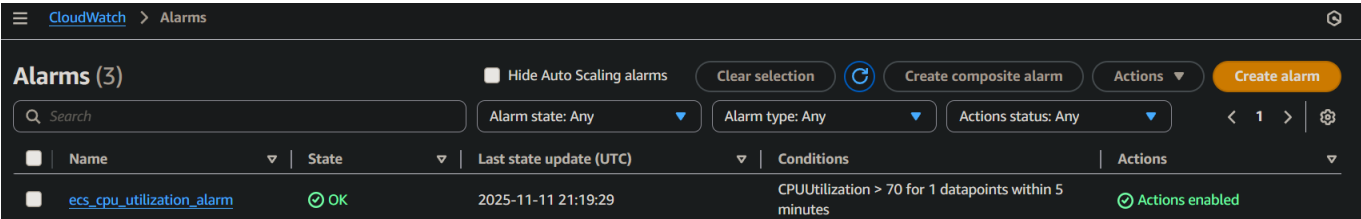
4. Set up monitoring and logging
 - Ensure that CloudWatch is collecting the right metrics.

CPU and memory utilization metrics for ECS.

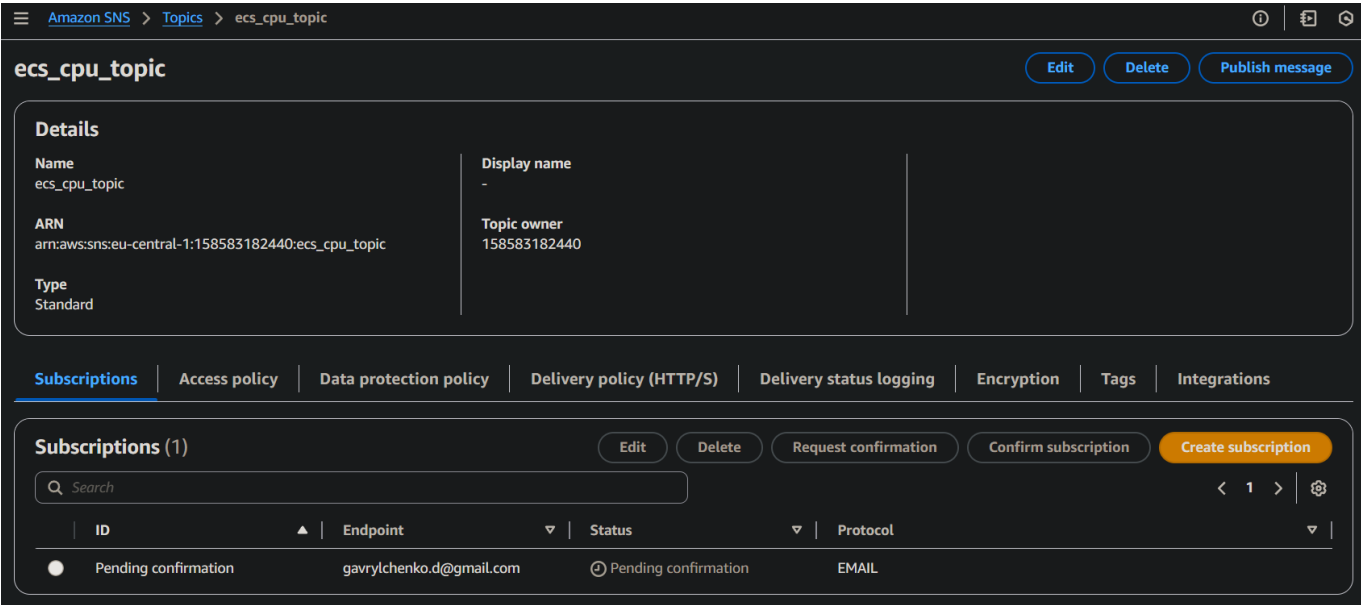


- Add SNS notifications for critical events.

Alarm in case when CPU usage is above 70%.



SNS topic for the alarm.



5. Optimize costs

- Set up AWS Budgets to control costs.

Notification of the monthly overbudget.

AWS Budgets: My Monthly Cost Budget has exceeded your alert threshold Inbox x

Inbox x



budgets@costalerts.amazonaws.com

to me ▼

10:30 PM (1 minute ago)



AWS Budget Notification

AWS Account 158583182440

November 11, 2025

Dear AWS Customer,

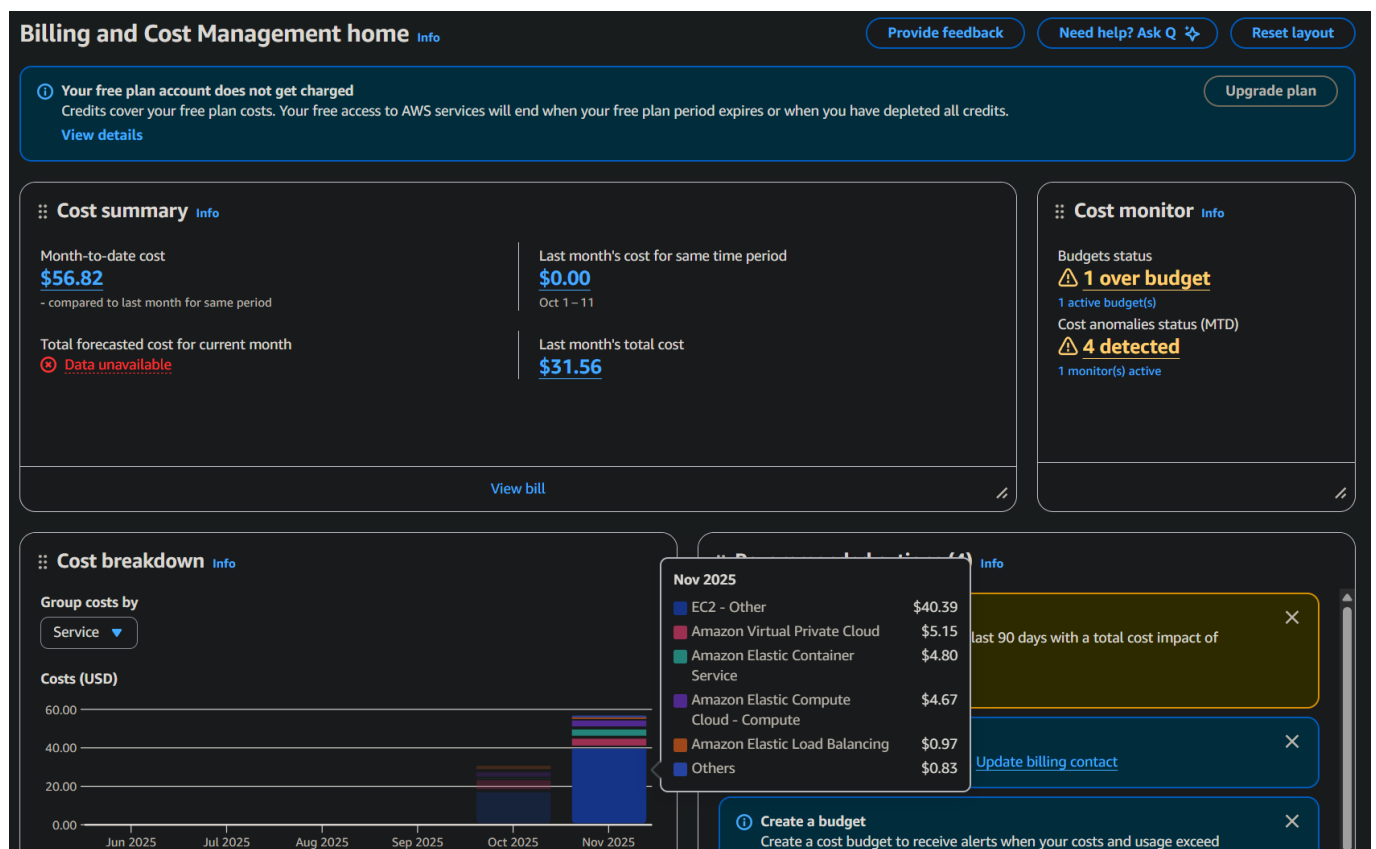
You requested that we alert you when the **actual cost** associated with your *My Monthly Cost Budget* budget **exceeds \$10.00** for the current month. The month **actual cost** associated with this budget is **\$56.82**. You can find additional details below and by accessing the AWS Budgets dashboard.

Budget Name	Budget Type	Budgeted Amount	Alert Type	Alert Threshold	ACTUAL Amount
My Monthly Cost Budget	Cost	\$10.00	ACTUAL	> \$10.00	\$56.82

[Go to the AWS Budgets dashboard](#)

- Use Cost Explorer to analyze the most expensive services.

The largest costs are associated with EC2 service, this can be explained by the presence of ALB in this project, as well as a large number of EC2 instances from previous homework that were stopped, but not deleted (in the stopped state, they also incur costs, although less than in the working state).

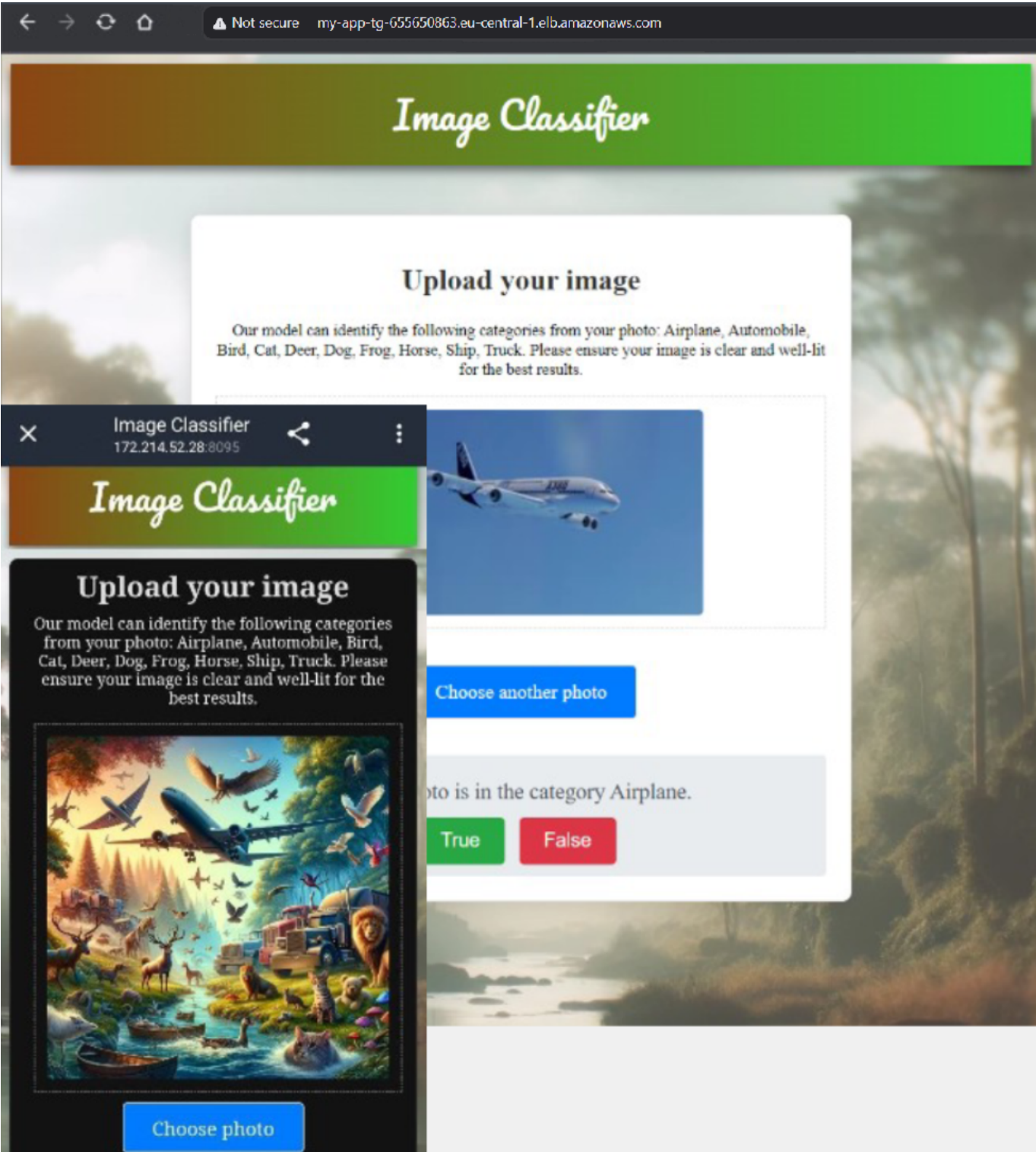


- ## 6. Test and verify system operation

- Ensure that your application is accessible at the intended URL.

The application is accessible under the link

http://my-app-tg-655650863.eu-central-1.elb.amazonaws.com/



- Perform load testing.

